

Share Lock

「一個安全、高效的檔案分享軟體」

網頁程式設計 期末專題 第十組

部屬網址：<https://www.sharelock.qzz.io>

原始碼：<https://github.com/whitebear13579/share-lock>

大綱

Outlines

01 開發動機

02 網站架構

03 使用套件

04 技術細節

05 主要介面展示

01 | 開發動機

為了解決與人共享檔案不能同時**兼具安全與快速**的痛點，
我們開發了 ShareLock 來解決這個問題。
以下是我們專案的特色：

- 流暢且美觀 UI / UX 體驗。
- 高度**可靠且安全**的系統架構設計。
- 有別於市面上其他檔案分享軟體，除了直接下載與使用 PIN 碼外的，還實現**綁定單一裝置 / 單一帳號**。

02 | 網站架構

網站頁面架構圖：



02 | 網站架構

API Route 架構圖：

```
/ API Route
├── auth
│   ├── login-history // 取得使用者登入紀錄
│   ├── record-login  // 向後端發送登入紀錄
│   └── session        // session 身分驗證
├── download
│   ├── issue-url      // 簽發一次性下載令牌
│   └── stream          // 串流檔案至使用者本機
├── files
│   ├── delete         // 刪除檔案
│   └── delete-all    // 刪除所有檔案
... 
```


02 | 網站架構

API Route 架構圖：

/ API Route	
files	
detail	// 取得檔案詳情 (擁有者)
download	// 檔案頁面管理下載請求
list	// 取得檔案列表
recent	// 紀錄最近使用的檔案
update	// 更新檔案狀態
notifications	
delete	// 刪除通知
list	// 取得通知列表
respond	// 通知回應 (檔案分享邀請)
...	

02 | 網站架構

API Route 架構圖：

/ API Route		
└── share		
└── bind-account	//	檔案帳號綁定
└── get-info	//	取得檔案詳情 (分享頁面)
└── verify-pin	//	驗證 PIN 碼
└── statistics		
└── overview	//	取得統計資料
└── storages		
└── confirm-upload	//	驗證配額、檢查較驗令牌
└── usage	//	儲存空間用量統計
└── validate-upload	//	驗證配額、生成較驗令牌
...		

02 | 網站架構

API Route 架構圖：

```
/ API Route
├── webauthn
│   ├── finish-assertion // 完成金鑰驗證整請求
│   ├── finish-registration // 完成金鑰註冊請求
│   ├── start-assertion // 開始裝置驗證
│   └── start-registration // 註冊密碼金鑰
```


03 | 使用技術

FRONTEND



Next.js



Tailwind CSS



Emfont WebFont



Lucide Icon



SimpleWebAuthn



Hero UI



GSAP



Frame Motion



IP API



Gravatar

BACKEND



Firebase



SimpleWebAuthn

OTHER



Node.js



Zeabur

04 | 技術細節

對於裝置綁定的實作，我們原先的計畫是採用：

- 裝置的 MAC 位址
- 廣告識別碼 (Content ID)

MAC address 與廣告識別碼是每個裝置獨有的，前者基本上不太會改變，而後者則可以根據使用者的需求自行重設，我們可以拿兩個資訊拿去做 SHA-256 來產生獨一無二的裝置金鑰。

這樣既能夠保證隱私與安全，也能完美實作裝置綁定功能。

04 | 技術細節

理想很豐滿，但現實卻很骨感。

- 基於安全性問題，瀏覽器環境無法取得真實 Mac 位址
- 非是同體系下的合作商，無法取得 Content ID

我們必須想一條全新的路來實作裝置綁定的功能。

— 這時，我們注意到了 WebAuthn 技術，或是他有更廣為人知的名稱，密碼金鑰。

04 | 技術細節

使用 PassKey 來驗證有幾個好處：

- 穩定、可靠且受 OS 管理
- 伺服器端不保存任何機敏資訊
- 判斷憑證設備類型便可達成綁定裝置的目的

```
91  
92     const backupEligible = regInfo.credentialDeviceType === "multiDevice";  
93     const backupState = regInfo.credentialBackedUp || false;  
94  
95     // block backup-eligible authenticators  
96     if (backupEligible) {  
97         return NextResponse.json(  
98             { error: "不允許使用具備份能力的驗證器" },  
99             { status: 400 }  
100        );  
101    }
```

[../webauthn/finish-registration/route.ts](https://github.com/duyquang101/webauthn/blob/main/src/routes/finish-registration/route.ts)



SOURCE CODE

04 | 技術細節

但這麼做也衍生了新的問題。

- 蘋果的 iOS / MacOS 上有個 APP 叫做 "密碼"，是原生提供的密鑰儲存工具。
- "密碼" APP 會透過 iCloud KeyChain 將密碼金鑰同步，導致蘋果使用者無法正常完成密鑰的註冊。
- 雖然有如 devicePubKey 之類的 WebAuthn Extension，在 RP 下能拿 per-device key 當作真正的裝置身分，但此技術現在仍不普及，不是所有瀏覽器都可以使用。

04 | 技術細節

我們最終的做法，是單獨為密碼 APP 開"後門"。

透過檢查驗證器的 AAGUID，來判斷嘗試儲存金鑰的平台是否為 Apple 的驗證器。

```
105      */  
106      const appleAAGUIDs = [  
107          "fbfc3007-154e-4ecc-8c0b-6e020557d7bd",  
108          "f24a8e70-d0d3-f82c-2937-32523cc4de5a",  
109          "adce0002-35bc-c60a-648b-0b25f1f05503",  
110          "00000000-0000-0000-0000-000000000000",  
111      ];  
112  
113      const isAppleAuthenticator = appleAAGUIDs.includes(aaguid);  
114  
115      // block backup-eligible authenticators, except for Apple devices
```

`../webauthn/finish-registration/route.ts`



SOURCE CODE

04 | 技術細節

我們使用了 GSAP 與 FrameMotion 實現了流暢的動畫效果。



04 | 技術細節



那麼，安全呢？

安全不能只是口說無憑，我們該如何證明我們的服務足夠安全？



在公開測試期間，我們取得了以下成果：

20+ 位

使用者參與公開測試

10 個

漏洞或 bug 被發現

*係指所有裝置都能穩定復現且足以影響正常使用

4 個

嚴重漏洞被發現

*係指可被有心人士利用之不當程式邏輯

100%

漏洞修復率

*係指已被合作夥伴通報之可疑漏洞



04 | 技術細節

- 原想實現如 Google Widevine 之類的 DRM 技術，部分分享模式下檔案只能供使用者線上存取。
- 實現 DRM 會牽扯到憑證簽發與檔案支援性的問題，現階段我們能力無法實現，最終作罷。
- 使用者把檔案下載後，確實我們就管不著了。但現今其實也依然沒有套「絕對安全」的數位內容保護機制。縱使是早已成熟的 Widevine 或 PlayReady，也難逃擷取卡或是用第三方裝置側錄等行為。

04 | 技術細節

展示後改善內容：

- 環境變數設定問題導致無法正常複製分享連結
- 大螢幕 Tab RWD 觸發寬度調整
- 部分按鈕 Spinner 大小調整
- 登出時組件的 race condition
- 刪除帳號後的路由跳轉問題

05 · 主要介面展示

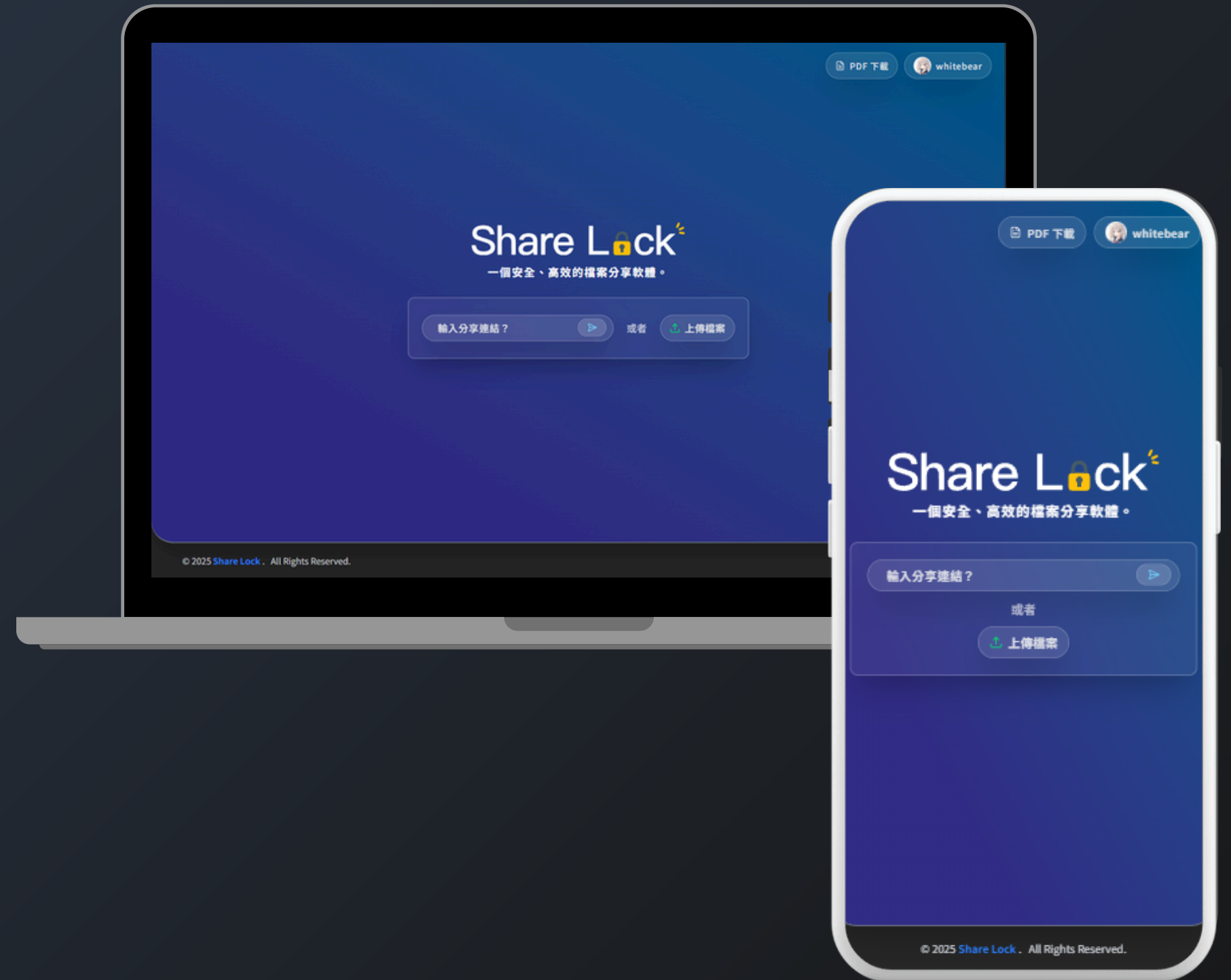
所有功能都能正常運作，但由於有大量動效或是條件渲染的組件
可能無法在簡報中完整呈現，建議實際至網站直接操作



05 | 主要介面展示

Share Lock 主頁面

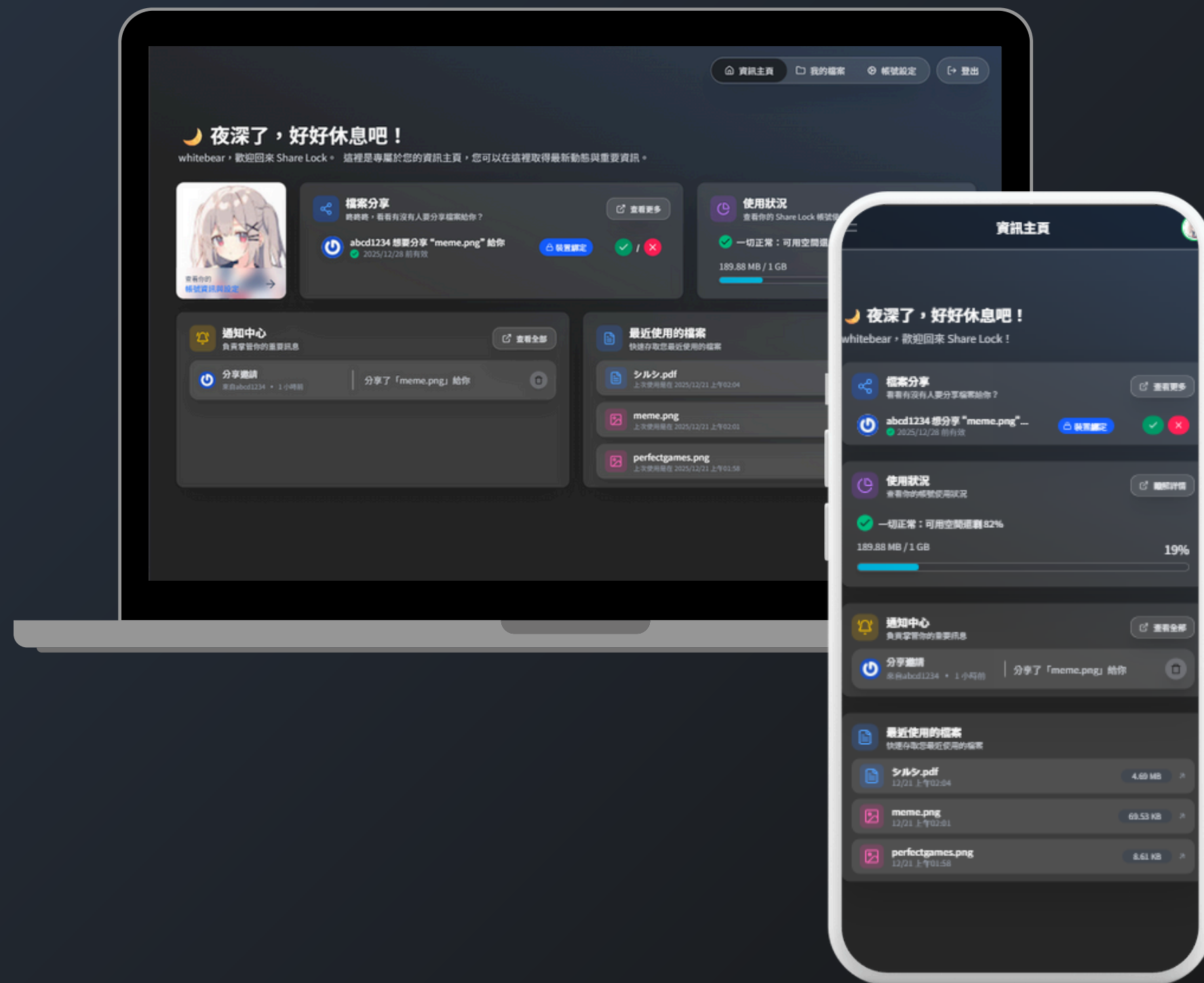
在Share Lock App中，我們大量的採用高斯模糊與圓角矩形的元素，並且使用 GSAP 套件繪製頁面之間的過場動畫，讓整體 UI / UX 具有現代感同時，提升使用者的使用體驗。



05 | 主要介面展示

帳號資訊主頁

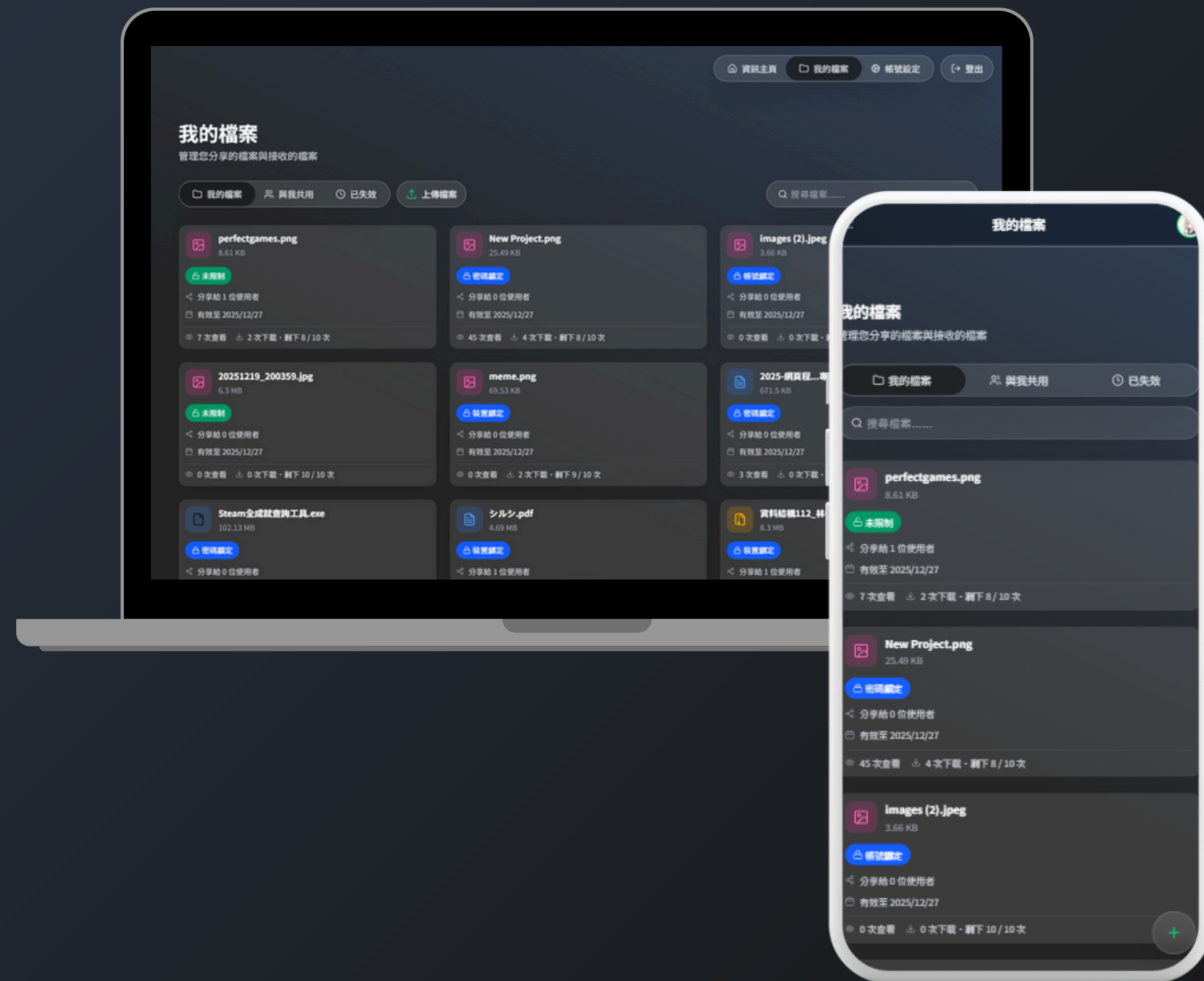
在帳號資訊主頁等一系列需要呈現大量資料的場景，我們除了延續以往的設計風格——「模糊」與「圓角」外，還大量的使用了卡片式的設計，讓重要資訊一目瞭然。



05 | 主要介面展示

我的檔案管理

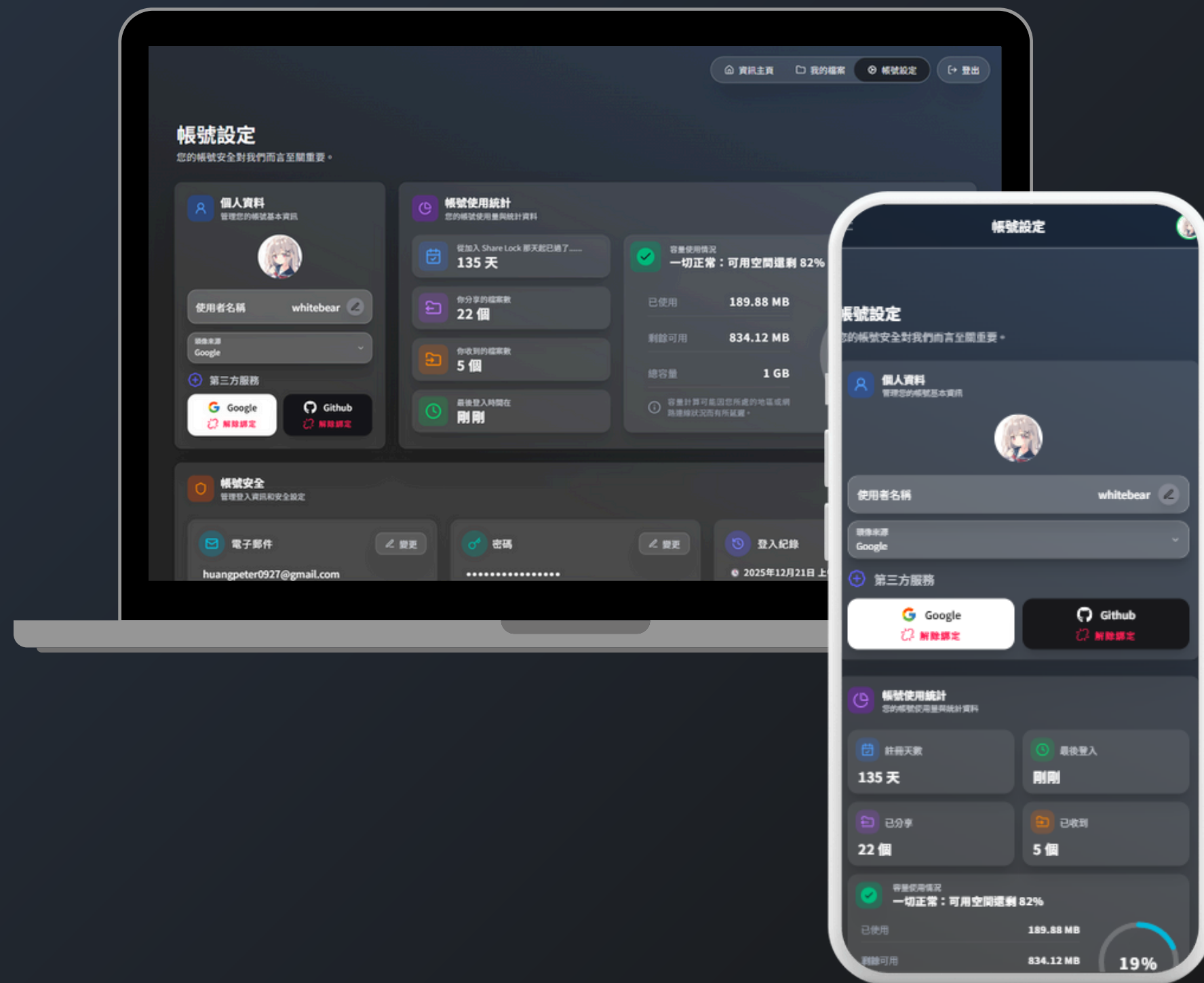
將檔案分為「我的檔案」、「與我共用」及「已過期」，讓使用者能快速根據需求篩選及尋找檔案。此外，也延續了卡片式設計的風格，將所有的檔案以卡片式排列呈現，方便使用者管理。



05 | 主要介面展示

帳號設定

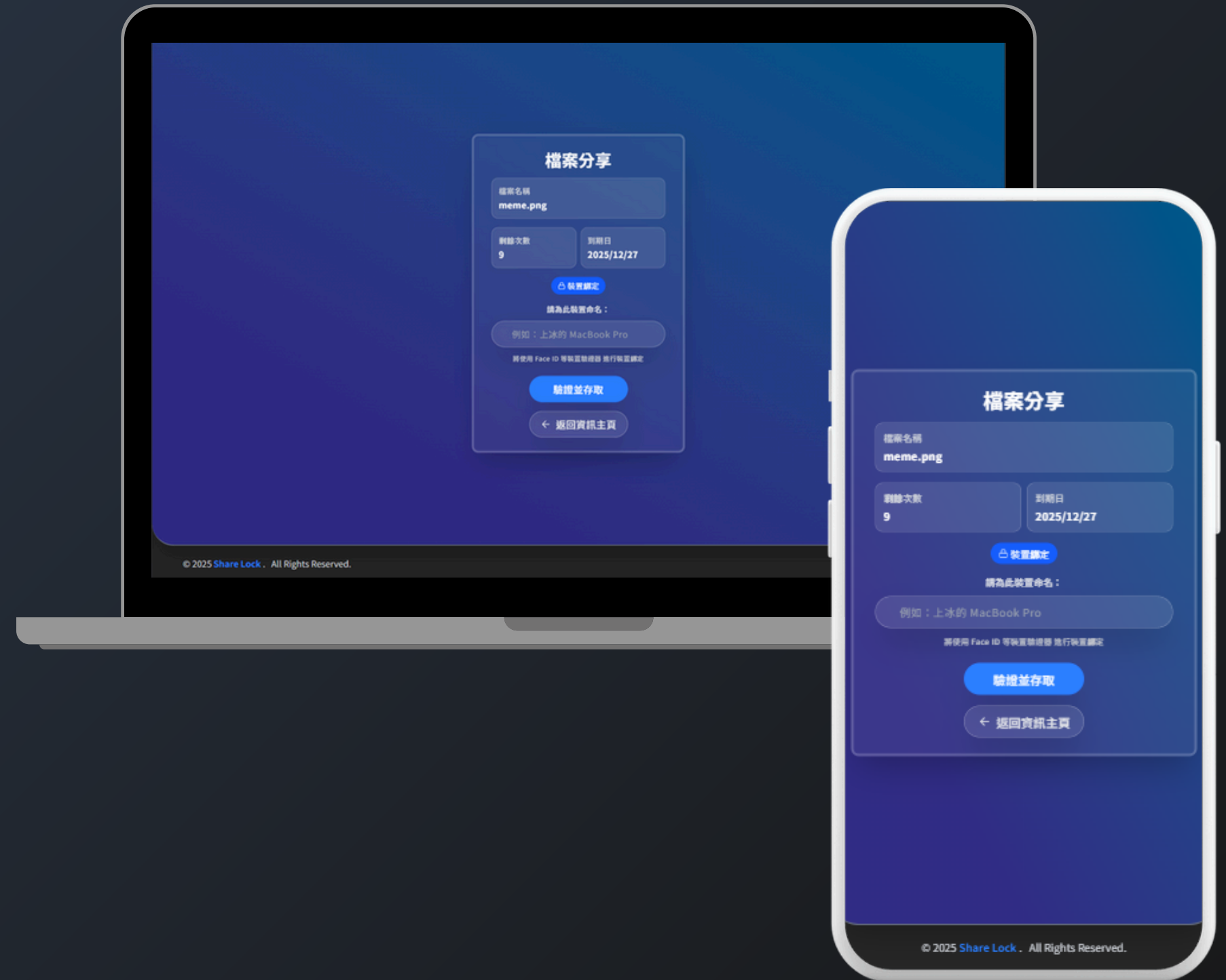
延續主視覺架構，以卡片式形式呈現使用者的資訊與相關統計資料。



05 | 主要介面展示

檔案分享頁面

被分享者能依據所持有的分享代碼連結到動態生成的檔案分享路由。在這裡，被分享者需要先完成分享者設定的特定步驟（如綁定裝置、輸入 PIN 碼），並經由後端驗證，才能進行檔案下載。



第十組小組成員與分工表



高莉芸

40%

動效設計
架構規劃
UI 設計
系統測試



黃翊宏

40%

動效設計
架構規劃
UI / UX 設計
API、組件實現

滲透測試 (與其他合作夥伴協同進行)



張楚珮

20%

UI 設計
系統測試



造訪儲存庫



前往Share Lock



Demo 影片